https://github.com/xreejal/COMP590-LAB01/tree/main/hwsec-course/lab-cacheattacks
Use UNC VPN if off campus
CPU assignments:
Group 8: 8, 24

There is no checkoff for Part 1. This part is graded manually based on the following submitted materials
under the "Part1-Timing" directory: code, the named question text files, and a pdf file.
• Code. Your code should be able to reproduce the histogram you submitted. Due to noise, we will run
your code multiple times (around 5 times) and grade based on the best results. You should feel
comfortable to submit your code as long as it can generate the expected results most of the time.
• The contents of 1.2.txt, 1.4.txt, and 1.5.txt.
• A pdf named solutions.pdf, containing your histograms from part 1.5.. Note that this pdf file will also
be used by you to provide answers for the questions in Part 2 and Part 3.

Discussion Question 1: Caches in modern processors are generally set-associative. Use the commands above to fill in the blanks in the following table. The L1 line size has been filled in for you. Record the answers in `1.2.txt`.

Hint: You should be able to directly obtain the information for the first 3 columns using commands above and you will need to derive the number of sets from the previous columns. Raw latency can be obtained from the WikiChip document.

1.2 txt:

	Cache Line Size	**Total Size**	**Number of Ways (Associativity)**	**Number of Sets (total size/(line size*associativity))**	**Raw Latency**
L1	64 bytes	32 Kib	8	64	4 cycles
L2	64 bytes	1 MiB	16	1024	12
L3	64 bytes	11MiB	11	16,384	42

If we are attempting to calculate the total size referring to the system sum, that would be:

Discussion Question 2: How many cache lines do you need to access to fill up the entire L2 cache and L3 cache respectively?

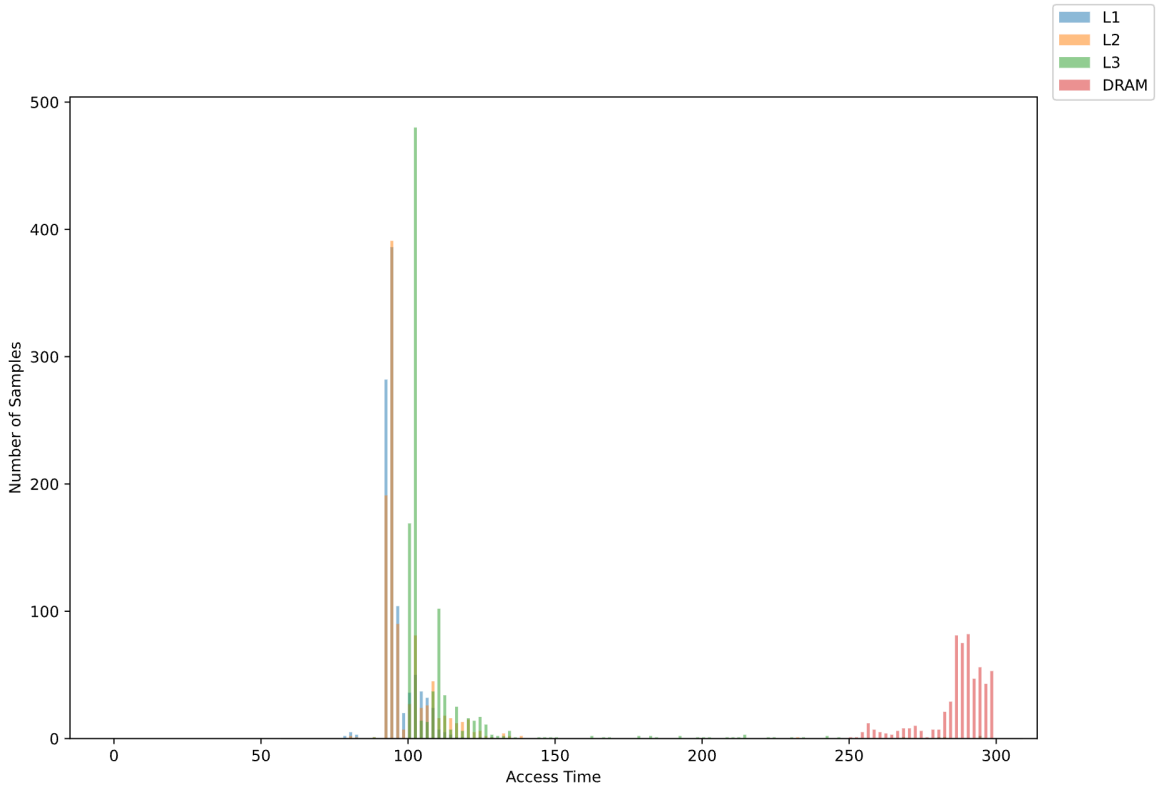
L2:
Size: 1 MB = 1,048,576 bytes
Line Size: 64 bytes
# of cache lines to fill L2 = 1,048,576 / 64 = 16,384 cache lines

L3:
L3 Size: 11 MB = 11,534,336 bytes
Line Size: 64 bytes
# of cache lines to fill L3 = 11,534,336 / 64 = 180,224 cache lines

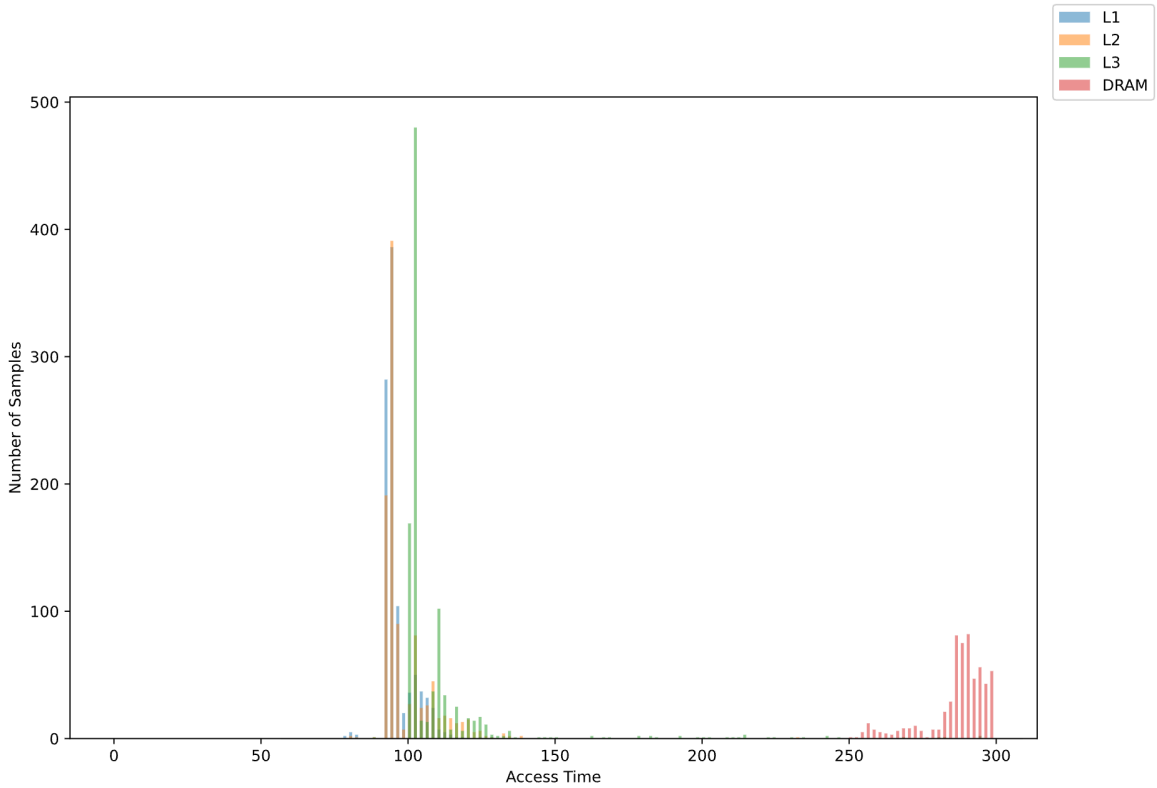
1.4.txt
Exercise 3: Fill in the code in main.c to populate the array dram_latency and report the medium DRAM latency. Compare your results with the reference implementation.

Reference med DRAM latency: 340
Our DRAM latency: 342

1.5.txt


L1
L2
L3
DRAM
Number of Samples
Access Time
0
100
200
300
400
500
0
50
100
150
200
250
300

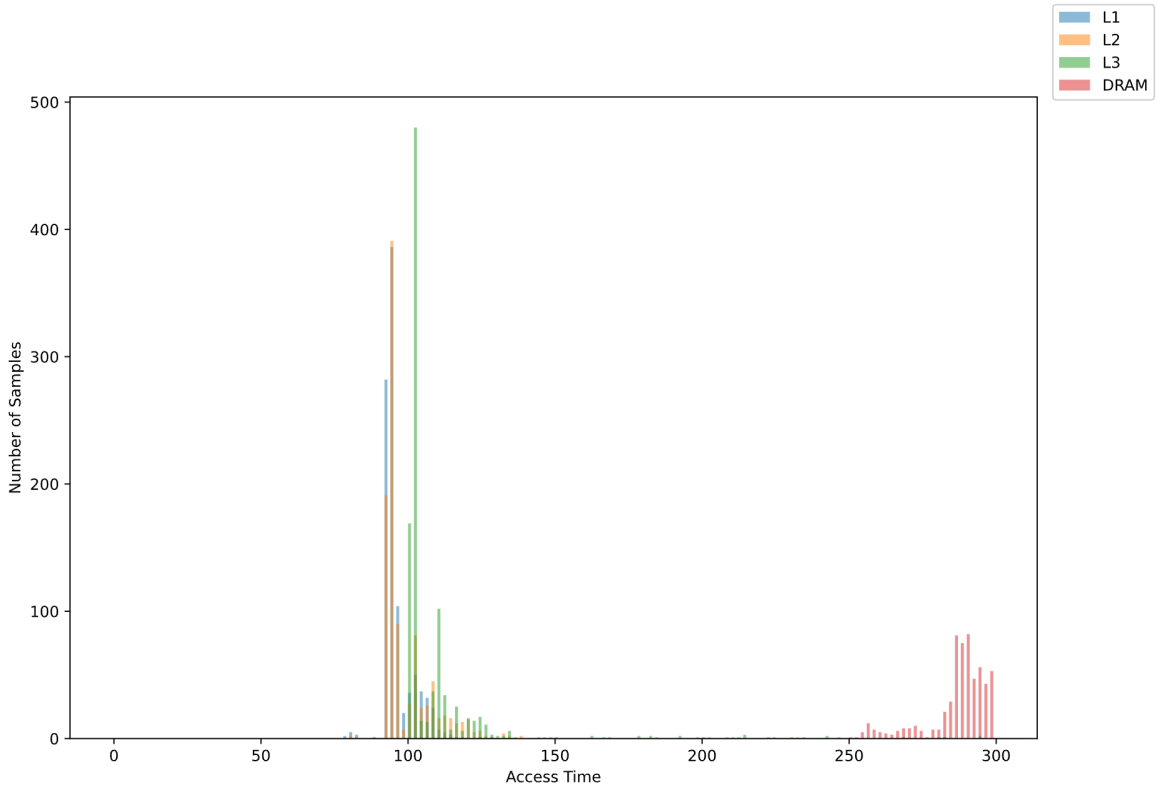
Reference



Our test

Discussion Question 3: Based on the generated histogram, report two thresholds, one to distinguish between L2 and L3 latency and the other to distinguish between L3 and DRAM latency.

Based on the histogram analysis:



Threshold to distinguish L2 from L3: 105 cycles
- L1/L2 cache hits typically show latencies around 90-100 cycles
- L3 cache hits typically show latencies around 100-120 cycles
- A threshold of 105 cycles effectively separates these two regions
- If latency < 105 cycles → L2 hit
- If latency >= 105 cycles → L3 or slower

Threshold to distinguish L3 from DRAM: 200 cycles
- L3 cache hits typically show latencies around 100-125 cycles
- DRAM accesses typically show latencies around 250-300 cycles
- A threshold of 200 cycles effectively separates these two regions
- If latency < 200 cycles → L3 hit
- If latency >= 200 cycles → DRAM access

Note: As mentioned in the lab instructions, L1 and L2 latencies are very similar on the Skylake architecture and are difficult to distinguish. However, {L1, L2} vs L3 vs DRAM are clearly distinguishable in our measurements.

Our performance:

```
        : L1   L2   L3   Mem
```

```
Minimum      :  110   110   116   296
Bottom decile:  110   110   116   338
Median       :  110   114   116   342
Top decile   :  112   122   132   352
Maximum      :  112   122   132   352
```

2.6 Communication Protocol (May need to be changed later)
Communication Protocol

1. Encode (Sender Actions for Bit 1 and Bit 0)

Bit 1: The sender “disturbs” the corresponding L2 cache set by traversing a linked list of cache lines allocated to that set.

Bit 0: The sender does nothing for that bit (the corresponding cache set remains untouched).

For an 8-bit integer, each bit is transmitted independently across sets 0–7, while set 8 acts as a valid-signal channel indicating that a transmission is in progress.

Each bit is transmitted for a fixed duration by looping over the cache traversal repeatedly (loops = 2,000,000), ensuring that the signal is detectable by the receiver.

2. Receive (Receiver Timing Measurement Strategy)

The receiver continually primes and measures the latency of accessing each set:

Priming: The receiver sequentially traverses the linked list of cache lines in a set to load them into the cache.

Probing: Immediately after priming, the receiver traverses the same set again and measures the access time using rdtscp.

The receiver first checks set 8 (the valid-signal channel) to detect whether a transmission is occurring. Only if this set shows high latency does the receiver begin accumulating samples for the payload bits (sets 0–7).

For each set, multiple measurements are taken (attempts = 120) with small delays in between to reduce noise and handle timing variations.

3. Decode (Threshold to Distinguish Bit 1 and Bit 0)

Each set has a threshold determined experimentally during an initialization phase:

The receiver measures the average access time for each set under normal conditions (cache hits) over multiple iterations (800 samples).

The threshold is set to twice the average hit time, i.e., cutoffs[s] = avg * 2.

During decoding, a bit is considered 1 if the measured latency for the corresponding set exceeds the threshold in a sufficient fraction of valid samples (> 70% of active samples). Otherwise, the bit is considered 0.

This thresholding method ensures that the receiver can distinguish between a disturbed (bit 1) and undisturbed (bit 0) cache line with high confidence, even in the presence of noise.

Summary of Components (per Section 2.2)

Component	Implementation
Encode	Disturb set if bit=1; do nothing if bit=0; use set 8 as valid signal
Receive	Prime and probe each set; measure latency; accumulate multiple samples
Decode	Compare measured latency against experimentally derived threshold (2×avg hit time); majority voting across samples for reliability